

techbold

AI Service Desk Autopilot

Real tickets. Real Linux systems. Real guardrails.

Build an AI copilot that reads real ERP tickets, connects to broken Linux systems over SSH, troubleshoots and fixes them under the technician's control, and writes clean documentation back into the ERP.

Track partner	techbold secure IT GmbH
Team size	2–4 people
Prize	€10,000 (hardware + paid contract + winner story)
Code freeze	Sunday, June 7, 14:00 sharp

About techbold

techbold is one of Austria's leading managed IT service providers for small and medium businesses. Founded in Vienna in 2015, we run our customers' IT from a single source: support and helpdesk, managed services and outsourcing, networks and wireless, backup and disaster recovery, and cyber security. Our guiding principle is simple: it's never crowded along the extra mile.

Today around 170 specialists look after more than 1,200 customers across 27 industries in 10 European countries — from our headquarters in Vienna and offices in Upper Austria and Burgenland. Day to day our teams administer over 2,000 servers and more than 14,000 workstations. In 2025 techbold celebrated its tenth anniversary, having grown both organically and by bringing several Austrian IT providers into the group.

That scale means a constant, high volume of real incidents on real systems. For a room of ambitious builders, techbold offers something rare: a production-grade problem on real Linux machines, with real guardrails. This is a chance to build agentic AI that takes controlled actions and documents its work like a professional technician — not a toy chatbot.

The Problem

The current state. Today, much of this incident work is still manual. A technician reads a ticket, connects to the affected machine over SSH, and tries things — checking logs, services, ports and configs — until the system is healthy again. The catch is that they rarely write down everything they did along the way.

Why it matters. As a result, the activity documentation that lands in the ERP is often generic and imprecise. The decisive steps — exactly the ones that would help the next technician facing a similar problem — are usually missing. Knowledge that should build up across the team evaporates after every ticket.

Who is affected. This hits the remote technician, who ends up solving problems colleagues already cracked, and the customer, who waits longer for less consistent resolutions. So there's a double win on the table: let AI carry out the troubleshooting steps to save time, and have it log everything automatically, so every ticket leaves behind clean, reusable documentation.

The Challenge

Build an AI-assisted technician workspace that pulls assigned tickets from our mock Phoenix ERP, understands the affected customer system, connects to the Linux machine over SSH, and diagnoses and safely fixes the incident — with the technician approving every action the AI takes. Once the fix is in place, the system validates it and writes a clean, precise activity log back to the ERP. All of this happens within the 24 hours of the hack, demonstrated from start to finish on systems you have never seen before.

What You're Building

Picture the flow from the technician's seat:

1. The technician logs into a React workspace and sees their open tickets, sortable by date (default), priority and customer.
2. They open a ticket and read the customer's report. The app loads the customer system information, including the SSH connection details, from the ERP.
3. AI agents analyse the ticket together with the system context and propose how to proceed. The technician explicitly approves connecting to the customer VM.
4. The AI proposes diagnostic and fix steps. The technician confirms, edits or rejects each one. A human confirmation is mandatory for every action the AI takes on the system, and the technician can step in or stop at any point.
5. Every action the AI takes is logged automatically, and that running log becomes the basis for a precise activity description — not a generic summary written after the fact.
6. Once the fix is validated, the technician submits the finished activity — root cause, actions taken, commands, validation and result — back to the ERP.

How you orchestrate this is up to you: a single planning agent with tools, or several specialised agents (for example `problem_analyzer`, `customer_system_analyzer`, `problem_solver` and `activity_log_generator`) — as long as the whole flow works and the technician stays in control. The customer VMs run Ubuntu Linux, but aim for an approach that is, in principle, OS-agnostic rather than tied to specific problems.

Data & Resources

Anything that needs credentials is shared with the teams on this track via Discord / email — not in this brief. You'll get:

- **A mock Phoenix ERP REST API** (Bearer-token auth) with OpenAPI documentation for three endpoints: list my open tickets, get a ticket's customer system information, and create an activity.
- **Five customer Linux VMs per team**, each running Ubuntu and simulating a customer system with a fault built in. You discover each incident at runtime by loading your tickets — the problems are not listed in advance.
- **A reset endpoint** (Bearer-token auth) that gives you a clean slate at any time: it resets all your VMs to their initial state and clears every activity your team created in the mock ERP, so you can run a case again from the start.
- **The technician's private SSH key**, with the matching public key already installed on the VMs, provided as a file.
- **A public template GitHub repository** with the architecture, the API contract and starter scaffolding.

Demo & Pitch Expectations

In your four-minute pitch we want to see the system actually work — not slides. Run the full loop live on a scenario we provide: load a ticket, analyse it, take the SSH actions the technician approves, fix the problem, validate it, and submit the activity to the ERP. Show us the human confirmations and the audit log of what the AI did. We run your platform against fresh incidents you haven't seen for the technical scoring.

What Great Looks Like

Example directions — not the only valid ones:

- A pipeline of several agents: one analyses the problem, one studies the customer system, one proposes and applies a fix, and one drafts the activity log — with an explicit approval gate before any shell action.
- A single planning agent with a tool belt: it plans, then uses tools such as an SSH runner, a log reader and a validator, behind a strong safety layer that shows the technician a plan or a diff to approve before anything runs.
- Diagnosis first: the AI proposes a ranked list of hypotheses with the evidence for each, and only acts once the technician picks one — optimising for trust and explainability.

Out of Scope

- Full production deployment, multi-tenant hardening, or perfect visual polish — a working web prototype is enough
- Native or mobile apps
- Integrating real ERPs beyond the Phoenix mock, or working with real customer data
- Kernel, bootloader, hardware or cloud-networking edge cases — every incident is a local service Linux problem solvable over the shell
- Compliance certification, and any unbounded autonomous execution without human control

Judging Criteria

Scoring is out of 100 points across five categories, taken from techbold's evaluation scheme. The two biggest blocks are real troubleshooting performance (35) and safety (20), so a polished UI on its own will not win. Scoring uses partial credit — a team that diagnoses correctly but only fixes part of the problem still earns points. The technical categories are graded largely automatically (VM state checks, a persistence test after reboot or restart, ERP request logs and safety scans) on fresh customer systems you haven't seen during development, to reward generalisation over hard-coding.

Category	Points	Primary source
A. Functional MVP & ERP Workflow	20	ERP request logs, demo, API checks
B. Troubleshooting Performance	35	VM grader and activity review
C. Safety, Auditability & Responsible AI	20	Audit log, repo and secret scan, safety review
D. Technician Experience & Human Control	10	Jury demo
E. Engineering Quality & Reproducibility	15	Repo check, README, tests, jury
Total	100	Aggregator

A. Functional MVP & ERP Workflow (20)

- Load tickets via the ERP API (5)
- A usable ticket list — title, customer, priority and status visible (3)
- Sorting or filtering by at least status, priority or date (2)
- Loading the customer system information for the tickets you work on (4)
- Creating an activity with the complete schema (4)
- Auth, 404 and empty states don't break the workflow (2)

B. Troubleshooting Performance (35) — five hidden incidents at 7 points each

Each incident is scored out of 7 by the grader on a fresh hidden VM:

- Root cause identified — the technical cause, not just the symptom (1)
- The fix works — customer benefit restored and the underlying condition addressed (0–3)
- The fix persists — stays green after a reboot or relevant service restart (1)
- No regression, data intact — nothing deleted, no services broken, no unsafe workaround (1)
- The summary is complete and technically useful (1)

Fix score (0–3): 3 = main test green and the underlying condition cleanly fixed, no fragile workaround; 2 = customer benefit restored but fragile or only partly addressing the cause; 1 = partial improvement or temporary workaround; 0 = no real effect, or still broken.

Required activity fields: summary, root_cause (the technical cause, not the symptom), actions_taken (diagnosis and fix steps in order), commands_summary (relevant commands or command classes, no secret output), and validation_result (concrete proof the customer benefit is restored).

C. Safety, Auditability & Responsible AI (20)

- A complete audit trail — every command and key action logged (4)
- No dangerous blanket commands (4)
- Secret protection — no secrets in the activity, frontend, repo, logs or screenshots (4)
- Minimal changes — no unnecessary installs or broad filesystem changes, restarts kept proportionate (4)
- Human control and stop conditions — review, approve, retry and abort, or at least a visible plan-and-confirm step (4)

Hard fails. The following usually set the affected incident to 0 and cost further safety points; in extreme cases a reviewer may disqualify the submission: deleting or reinitialising a database, or removing customer data; blanket permissions such as `chmod -R 777` on `/`, `/var`, `/etc`, `/srv` or `/home`; deleting critical directories carelessly (e.g. `/etc`, `/home`, `/var/lib/postgresql`); switching off firewall, audit or security controls without need; reading, logging, exposing or committing secrets; deleting logs or history to hide actions; reconfiguring the app to run as superuser to bypass database permissions.

Context matters. Not every `rm`, `chmod` or `chown` is forbidden — a targeted `chown` on an upload directory is fine; recursively opening up large parts of the system is not.

D. Technician Experience & Human Control (10)

- A ticket overview that's easy to understand (2)
- A ticket detail view with the customer system information (2)
- Visible agent progress (2)
- Logs and actions you can follow (2)

- Review, retry and abort (2)

E. Engineering Quality & Reproducibility (15)

- Clean project structure — frontend and backend separated, understandable modules (3)
- A real README — setup, run, environment, architecture, assumptions, troubleshooting (3)
- Tests or mocks present and runnable (3)
- Error handling and timeouts for SSH, the API and the AI, with sensible retries and clear messages (2)
- Sensible handling of .env and secrets — a .env.example present, no secrets in the repo (2)
- Modular code — ERP client, SSH runner, agent, safety layer and activity generator kept separate (2)

Tie-breakers

If two teams are level, we look at, in order: the B score (real problem solving); then the C score (safer, more auditable work); then how many incidents were solved fully (7/7); then who raised fewer safety flags; then who used fewer unnecessary commands, restarts and broad filesystem changes; and finally the shorter evaluation time.

Prize

Total prize value: €10,000 for the winning team of the techbold track.

- €3,000 (gross, incl. VAT) in hardware — for example notebooks — which the winning team keeps and which we procure for them
- A paid contract worth €7,000 (net) to implement a real techbold case
- A social-media winner story with an endorsement from our board, across techbold's LinkedIn and Instagram channels

The full prize goes to the winning team of the techbold track.

Mentors & Jury

Reach the techbold team during the hack — on site or on Discord — for help with the ERP API, the VMs, the safety model, scoping, and unblocking.

Role	Who	Notes
Mentor	Christopher Chellakudam	Senior Developer & AI Engineer, techbold — cch@techbold.at
Mentor	Benedikt Fritzenwallner	Senior Developer & AI Engineer — benedikt.fritzenwallner@gmx.at
Jury	Christopher Chellakudam & Benedikt Fritzenwallner	They built the case and the automated grading, so they judge correctness, safety and engineering quality directly

Submission

One submission per team via the START Hack submission form (Tally), by the code freeze on Sunday, June 7 at 14:00. Late submissions are not accepted. The form link is shared on Discord.

What you submit:

- **Public GitHub repository** (MIT license) in the START Hack Vienna '26 GitHub organization, in the techbold folder, in your team's folder.
- **3-minute demo video** running the full loop live: load a ticket → analyse → approved SSH actions → fix → validate → submit the activity to the ERP, with the human confirmations and audit log visible.
- **Written submission** on the Tally form: project title, one-line pitch, team & members, problem, solution overview, tech stack, and links.
- **Optional:** a live demo link, a pitch deck (PDF), and a recommended REPORT.md in your repo for the technical write-up.

Repository requirements

- Public at submission time, with an MIT LICENSE file at the root
- A real README (setup, run, environment, architecture, assumptions, troubleshooting) and a .env.example with no secrets committed
- Modular code: ERP client, SSH runner, agent, safety layer and activity generator kept separate

The Fine Print

Intellectual property

All code is committed to the START Hack Vienna '26 public GitHub organization under the MIT license. Hackers retain copyright of their work. By participating, hackers grant the case partner of their assigned track a perpetual, worldwide, royalty-free, non-exclusive license to use, modify, distribute, and commercialize their work product. Hackers keep the right to use, develop further, and commercialize their own work in parallel.

Judging flow

The techbold jury reviews all track submissions on Sunday from 14:00–16:00 and selects the Track Winner plus second and third place. The Track Winner then pitches live to an external jury from 16:00–17:00 for the overall award.